Computational Physics: Stochastic Methods and Path Integrals UCSD PHYS 142/242
Assignment 1 Due: Monday, October 19, 2026, 8:00pm

Policies

- You are free to collaborate on all of the problems, subject to the collaboration policy stated in the syllabus.
- You should submit all code used in the homework. You are free to use Python, C/C++, Julia, or any other code **within reason** as long as you comment your code such that the TA can follow along and run it without any issues.
- Please submit your report as a single .pdf file to Gradescope under "Assignment 1". **In the report, include any images generated by your code along with your answers to the questions.** For instructions specifically pertaining to the Gradescope submission process, see `https://www.gradescope.com/get_started#student-submission`.
- Please submit your code, together with the `results.json` file it writes (see *Autograded results* at the end), to Gradescope under "Assignment 1 Code", as individual files or a single .zip archive. That submission is autograded and worth 20 of the 50 points; the other 30 are graded from your report.
- End your report with an AI use appendix, as described in the syllabus: which tools you used and for what, the prompts you used (or a share link), and what the tool got wrong and how you caught it. "I did not use AI tools on this assignment" is a complete answer.
- You will walk through your code for this assignment at Oral Checkpoint 1 (Lab 07, Tuesday, October 20). Bring a laptop that can run it.

1 Harmonic Oscillator Path Integral [50 Points]

Relevant materials: Lectures 3–6

Consider the harmonic oscillator with Lagrangian,

$$L(x, \dot{x}) = K(\dot{x}) - V(x) = \frac{1}{2}\dot{x}^2 - \frac{1}{2}x^2, \tag{1}$$

where x is the position of the oscillator and $\dot{x}$ is its velocity. This is expressed in units where the mass $m = 1$, the angular frequency $\omega = 1$, and $\hbar = 1$, so the classical oscillator period is $T_0 = 2\pi$.

We will use the discrete approximation to the path integral for the harmonic oscillator, where the time step is $\epsilon = \Delta t = T_0/128$. The electron position is also discretized into $N_D + 1$ possible points, $x_0 = -4, x_1, x_2, \ldots, x_{N_D} = +4$, where $N_D = 600$. The initial probability amplitude (sometimes called the wavefunction) of the electron is a Gaussian centered at x_{start},

$$\Psi_0(x) = \left(\frac{\alpha}{\pi}\right)^{1/4} \exp\left(-\frac{\alpha}{2}(x - x_{\text{start}})^2\right), \tag{2}$$

where $\alpha = 2$ and x_{start} is set by your Triton Student Number (TSN: nine digits starting with 2, which replaces the PID),

$$x_{\text{start}} = (-1)^s \left(0.5 + \frac{d}{200}\right), \tag{3}$$

where d is the last two digits of your TSN (00 to 99), and s is the third-to-last digit of your TSN (0 to 9). The sign of x_{start} is positive if s is even and negative if s is odd. For example, a TSN of 212345678 gives $x_{\text{start}} = +(0.5 + 78/200) = +0.89$, and 212345123 gives $x_{\text{start}} = -(0.5 + 23/200) = -0.615$. State your x_{start} at the top of your report. The amplitude can be represented as a vector ψ_0 with $N_D + 1$ components, $\psi_0 = (\Psi_0(x_0), \Psi_0(x_1), \ldots, \Psi_0(x_{N_D}))$. We recommend using complex NumPy arrays, e.g. `np.array([1+2j, 3+4j])` and `np.zeros((10, 10), dtype=np.complex64)`.

Problem A [10 points]: Calculate the propagator matrix $\mathcal{K}_{8\epsilon}$ for a time period $T_0/16 = 8\epsilon$ (8 time steps) built from the elementary propagator matrix $\mathcal{K}_\epsilon$ for a single $\epsilon = \Delta t = T_0/128$ time step.

Recall we gave the general form of the propagator in lecture as

$$\mathcal{K}(x_b, t_b; x_a, t_a) = \left(\frac{m\omega}{2\pi i\hbar \sin(\omega(t_b - t_a))}\right)^{1/2} \exp\left(\frac{im\omega}{2\hbar \sin(\omega(t_b - t_a))}[(x_a^2 + x_b^2)\cos(\omega(t_b - t_a)) - 2x_a x_b]\right) \tag{4}$$

where x_a and x_b are the initial and final positions, respectively, and t_a and t_b are the initial and final times, respectively.

Use NumPy to print the matrix (default truncated output) and copy the (truncated) output into your report. Note by default if `K_8eps` is a large NumPy array, `print(K_8eps)` prints the first 3 and last 3 elements along each axis.

Hint: *The elementary propagator matrix $\mathcal{K}_\epsilon$ is an $(N_D + 1) \times (N_D + 1)$-dimensional complex matrix that time evolves the state ψ by one time step, and*

$$\mathcal{K}_t = (\Delta x)^{N-1} \mathcal{K}_\epsilon^N \tag{5}$$

time evolves the state by N time steps, where $N\epsilon = t$.

Problem B [10 points]: Evolve the probability amplitude of the electron for one full period $T_0 = 2\pi$ (128 time steps) and measure its mean position $\langle x \rangle$ as a function of time. Make a graph showing $\langle x \rangle$ versus time t. Label the axes.

Hint: *Recall*

$$\langle x \rangle = \int x P_t(x) dx \tag{6}$$

where $P_t(x) = |\Psi_t(x)|^2$ is the probability density function at time t.

Problem C [10 points]: Calculate the mean energy $\langle E \rangle$, mean kinetic energy $\langle K \rangle$, and mean potential energy $\langle V \rangle$ as a function of time. Make one graph showing all three with a legend labeling them.

Hint: *Recall $E = K + V$ and for the mean value of V, we have*

$$\langle V \rangle = \int \frac{1}{2} x^2 P_t(x) dx. \tag{7}$$

For the mean value of K, there are a few different ways to calculate it. The simplest uses the kinetic energy operator (Lecture 2), but rewrites it in terms of the first derivative of the wavefunction using integration by parts:

$$\langle K \rangle = \int \Psi_t^*(x) \left(-\frac{\hbar^2}{2} \frac{\partial^2}{\partial x^2} \right) \Psi_t(x) dx = \frac{\hbar^2}{2} \int \left| \frac{\partial \Psi_t}{\partial x} \right|^2 dx \tag{8}$$

Problem D [10 points]: Calculate the time evolution of the probability amplitude at times $mT_0/16$ for $m = 1, \ldots, 8$. Make a single graph showing the probability amplitudes initially and at those eight times, with a legend labeling them. Optional: Superimpose the potential $V(x)$ and amplitudes at those eight times on the same graph as the probability density function $P_t(x) = |\Psi_t(x)|^2$ at those eight times.

Problem E [10 points]: Animate the time evolution of the probability amplitude over the full time period $T_0 = 2\pi$. Each frame should correspond to one time step of $\epsilon = T_0/128$ (so $128 + 1$ frames total). Save the animation as a `.gif` or `.mp4` file.

Problem F [10 points]: *Optional for bonus points.* Without renormalizing the state, compute the norm $\mathcal{N}(t) = \Delta x \sum_i |\Psi_t(x_i)|^2$ over one full period in three cases: (i) $\epsilon = T_0/16$ with $N_D = 600$; (ii) $\epsilon = T_0/512$ with $N_D = 600$; and (iii) $\epsilon = T_0/128$ with $N_D = 150$. Before running them, predict which will fail. Then plot $\mathcal{N}(t)$ for all three on a logarithmic scale.

Explain what you see in terms of how rapidly $\mathcal{K}_\epsilon(x_b, x_a)$ oscillates as a function of x_a, compared with the grid spacing Δx. Estimate the smallest time step that works for $N_D = 600$, and how it changes when you double N_D.

Hint: *The propagator from Problem A is exact for any time step, so the only approximation in this method is the discretization of x.*

Autograded results

Your code should write a file `results.json` containing the following fields, and you should upload it with your code. Any language can write JSON; the Python snippet below shows the format.

Field	Contents
`tsn`	Your TSN, e.g., `"212345678"`
`x_start`	Your x_{start}
`K8eps_row`	Row $N_D/2 = 300$ (the row for $x = 0$) of $\mathcal{K}_{8\epsilon}$, as real parts `re` and imaginary parts `im` (601 values each)
`x_mean`	$\langle x \rangle$ at $t = n\epsilon$ for $n = 0, 1, \ldots, 128$ (129 values)
`V, K, E`	$\langle V \rangle$, $\langle K \rangle$, and $\langle E \rangle$ at the same 129 times
`P`	$P_t(x_i) = \lvert\Psi_t(x_i)\rvert^2$ on the grid at $t = mT_0/16$ for $m = 0, 1, \ldots, 8$ (9 lists of 601 values)

```
import json

results = {
    "tsn": "212345678",
    "x_start": x_start,
    "K8eps_row": {"re": K_8eps[300].real.tolist(),
                  "im": K_8eps[300].imag.tolist()},
    "x_mean": x_mean.tolist(),
    "V": V.tolist(), "K": K.tolist(), "E": E.tolist(),
    "P": [P_t.tolist() for P_t in P_list],
}
with open("results.json", "w") as f:
    json.dump(results, f)
```

The autograder awards 4 points for Problem A, 5 for B, 6 for C, and 5 for D, using your own x_{start}; the rest of those problems, and all of Problem E, are graded from your report. When you submit, Gradescope immediately tells you whether it could read your `results.json` and whether its parameters match your TSN. Your autograded score appears after the due date, and you may resubmit as often as you like before then.